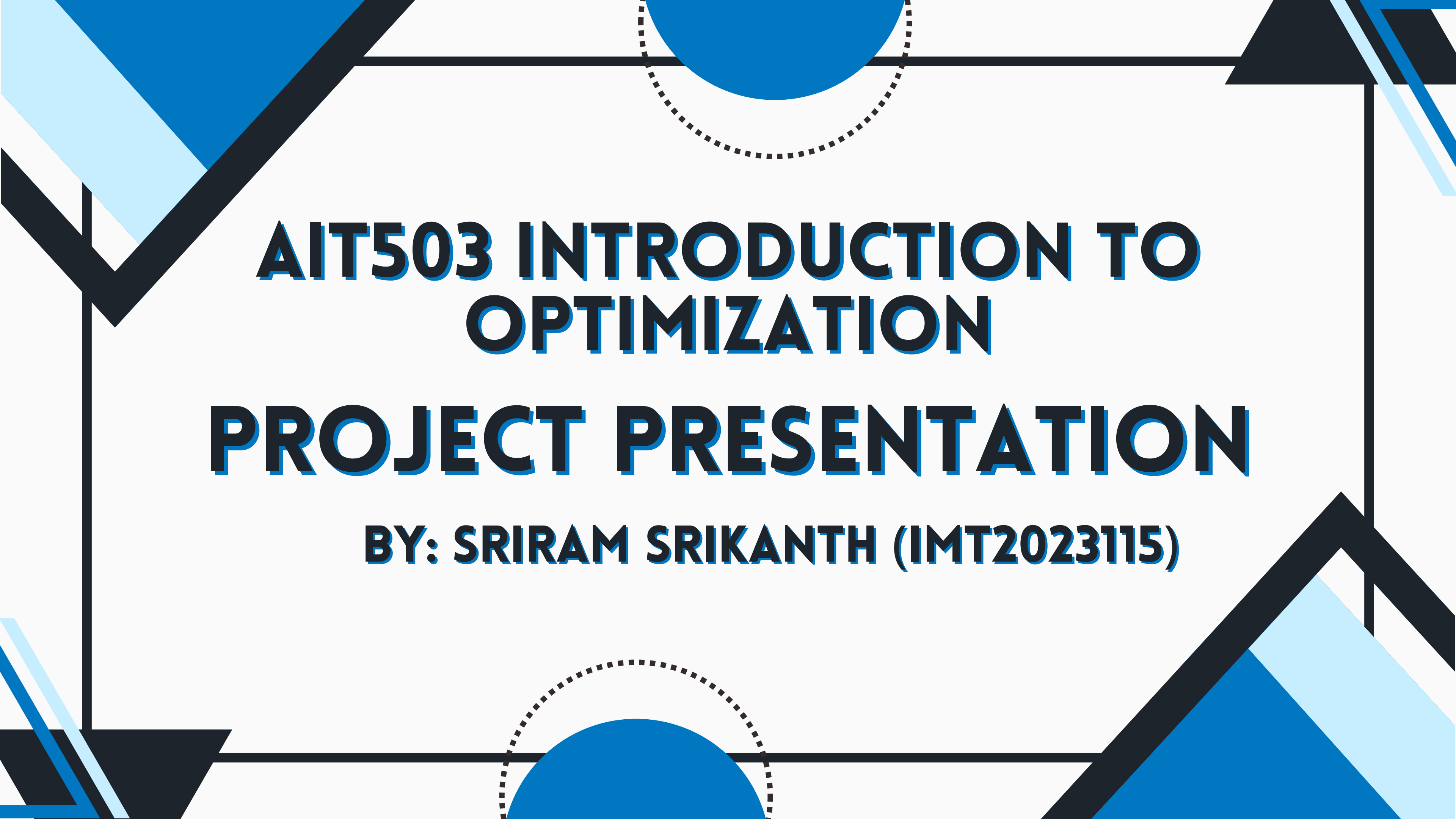




AIT503 INTRODUCTION TO OPTIMIZATION

PROJECT PRESENTATION

BY: SRIRAM SRIKANTH (IMT2023115)

DATASET: IPL BALL-BY-BALL DATASET

Feature Name	Data Type	What it means?
id	int	Uniquely identifies a match in IPL.
Innings	int	The current innings number (1 or 2, sometimes 3/4 in special cases).
Overs	int	The over number within the innings (0–20 in T20).
BallNumber	int	Ball number within the over (1–6, extra balls possible).
Batter	string	Name of the batter on strike.
Bowler	string	Name of the bowler delivering the ball.
NonStriker	string	Name of the batter at the non-striker's end.
FieldersInvolved	string	Name(s) of fielder(s) involved for the ball (separated by ,).
BattingTeam	string	Team name of the batting side.
BowlingTeam	string	Team name of the bowling side.
city	string	City where the match is played.
venue	string	Stadium/venue name.
umpire1	string	Name of the first on-field umpire.
umpire2	string	Name of the second on-field umpire.
BatsmanRun	int	Runs scored by the batter off the ball (0–6).
ExtrasRun	int	Runs given as extras (wides, no-balls, leg-byes, byes, penalty runs).
PlayerOut	boolean (0/1)	Binary indicator: 1 if the ball resulted in a player getting out, 0 otherwise.
ExtraType	string	Type of extra if any (e.g., Wide, No-ball, Leg-bye, Bye, Penalty); NULL / None if no extra. (Dependent on ExtrasRun > 0).
OutKind	string	Means by which batsman got out (e.g., Bowled, Caught, LBW, Run Out); NULL / None if not out. (Dependent on PlayerOut = 1).

DATA PREPROCESSING

- Preprocessed pre-existing features to build batsman, bowler and ball_by_ball profiles.
- Removed unnecessary features from the initial dataset.

FINAL CSV FILES

BALL_BY_BALL_PROFILE.CSV

BATSMAN_PROFILES.CSV

BOWLER_PROFILES.CSV

(REMOVED FIELDS-
FIELDERS_INVOLVED,
UMPIRE1,UMPIRE2, CITY)

BATSMEN_PROFILES.CSV

Feature Name	What it is	How we got it
Batsman	The unique name of the batter.	Aggregated from the <code>Batter</code> column in <code>main_df</code> .
Innings	Number of unique matches played.	Calculated by counting unique <code>id</code> values for each <code>Batter</code> .
TotalRuns	Total career runs scored.	Calculated by summing <code>BatsmanRun</code> for each <code>Batter</code> .
StrikeRate	Runs scored per 100 balls faced.	Calculated as <code>(mean of BatsmanRun) * 100</code> for each <code>Batter</code> .
AverageRuns	Average runs scored per dismissal.	Calculated as <code>(TotalRuns) / (Total times out)</code> . Used <code>1e9</code> for batters never out.
Fours	Total number of 4s hit.	Calculated by counting rows where <code>BatsmanRun == 4</code> for each <code>Batter</code> .
Sixes	Total number of 6s hit.	Calculated by counting rows where <code>BatsmanRun == 6</code> for each <code>Batter</code> .
NotOut	Total balls faced where the batter was not dismissed.	Calculated by counting rows where <code>PlayerOut == 0</code> for each <code>Batter</code> .
BallsFaced	Total number of balls faced.	Calculated by counting all rows (balls) for each <code>Batter</code> .
Role	Inferred batting role (opener, middle, lower).	Determined by the custom <code>infer_role</code> function based on <code>Overs.mean()</code> , <code>BallsFaced</code> , and <code>StrikeRate</code> .

BOWLER_PROFILES.CSV

Feature Name	What it is	How we got it
<code>Bowler</code>	The unique name of the bowler.	Aggregated from the <code>Bowler</code> column in <code>main_df</code> .
<code>Innings</code>	Number of unique matches played.	Calculated by counting unique <code>id</code> values for each <code>Bowler</code> .
<code>TotalWickets</code>	Total career wickets taken.	Calculated by counting rows where <code>PlayerOut == 1</code> for each <code>Bowler</code> .
<code>TotalRunsConceded</code>	Total runs given up (batting + extras).	Calculated by summing <code>BatsmanRun</code> + <code>ExtrasRun</code> for each <code>Bowler</code> .
<code>EconomyRate</code>	Average runs conceded per over (6 balls).	Calculated as $(\text{TotalRunsConceded} / \text{BallsBowled}) * 6$.
<code>AverageRunsConceded</code>	Average runs conceded per wicket taken.	Calculated as $(\text{TotalRunsConceded} / \text{TotalWickets})$. Set to 0 if no wickets were taken.
<code>StrikeRate</code>	Average balls bowled per wicket taken.	Calculated as $(\text{BallsBowled} / \text{TotalWickets})$. Set to 0 if no wickets were taken.
<code>BallsBowled</code>	Total number of balls bowled.	Calculated by counting all rows (balls) for each <code>Bowler</code> .

PROBLEM STATEMENT

To find an optimal ordering of batsmen from the given set of batsmen such that the runs made is maximised.

This is a combinatorial maximisation problem.

The easy and hard methods I have chosen to solve this optimisation problem are:

- Greedy method
- Dynamic Programming method

VARIABLES

$\mathcal{B} = \{b_1, b_2, \dots, b_{11}\}$: Set of available batsmen

$\mathcal{W} = \{B_1, B_2, \dots, B_m\}$: Set of bowlers

$K = \{1, 2, \dots, 20\}$: Set of overs

i, j : Indices for batsmen ($i, j \in \{1, \dots, 11\}$)

k : Index for over number ($k \in K$)

w : Number of wickets lost ($w \in \{0, 1, \dots, 10\}$)

$\sigma = [\sigma_1, \sigma_2, \dots, \sigma_6]$: Given bowling order

$\pi = [\pi_1, \pi_2, \dots, \pi_6]$: Batting order (permutation of $\mathcal{B}$)

π_p : Batsman at position p in the batting order, where $p \in \{1, \dots, 6\}$

OBJECTIVE FUNCTION

Input: set of batsmen, probability matrix of probability of bowler bowling in ith over, list of bowlers in the ordering they are going to bowl, the previously mentioned datasets

Objective function:

$$\mathbb{E}[\text{Total Runs} \mid \pi, P] = \sum_{k=1}^{20} \sum_{j=1}^m P_{j,k} \cdot \sum_{b=1}^6 \sum_{r=0}^6 r \cdot P(R = r \mid \pi_{\text{striker},k,b}, Bo_j)$$

subject to:

$$w_k = w_{k-1} + \sum_{b=1}^6 P(W \mid \pi_{\text{striker},k-1,b}, \sigma_{k-1}) \quad \forall k \geq 2$$

$$w_1 = 0 \quad (\text{no wickets initially})$$

$$w_k \leq 10 \quad \forall k \quad (\text{all out condition})$$

$$P_{j,k} \geq 0 \quad \forall j \in \{1, \dots, m\}, \forall k \in K$$

Each bowler can bowl at most 4 overs (T20 rule)

$$\sum_{k=1}^{20} P_{j,k} \leq 4 \quad \forall j \in \{1, \dots, m\}$$

Bowling probability matrix: $P \in [0, 1]^{m \times 20}$ where P_{ij} is the probability that bowler i bowls over j

π is a permutation of $\mathcal{B}$

GREEDY ALGORITHM

Algorithm 1 Greedy Optimization Method

```
1: Input:  $\mathcal{B}$  (batsmen),  $\sigma$  (bowling order)
2: Output:  $\pi^*$  (batting order),  $R^*$  (expected runs)
3:  $\pi \leftarrow []$ ,  $\mathcal{R} \leftarrow \mathcal{B}$  (remaining batsmen)
4: for  $p = 1$  to 6 do
5:    $b^* \leftarrow \text{null}$ ,  $R_{\max} \leftarrow -\infty$ 
6:   for each  $b \in \mathcal{R}$  do
7:      $\pi_{\text{temp}} \leftarrow \pi \cup \{b\} \cup \text{HeuristicCompletion}(\mathcal{R} \setminus \{b\})$ 
8:      $R \leftarrow \mathbb{E}[\text{Total Runs} \mid \pi_{\text{temp}}, \sigma]$ 
9:     if  $R > R_{\max}$  then
10:        $R_{\max} \leftarrow R$ 
11:        $b^* \leftarrow b$ 
12:     end if
13:   end for
14:    $\pi \leftarrow \pi \cup \{b^*\}$ 
15:    $\mathcal{R} \leftarrow \mathcal{R} \setminus \{b^*\}$ 
16: end for
17: return  $\pi$ ,  $\mathbb{E}[\text{Total Runs} \mid \pi, \sigma]$ 
```

Algorithm 2 Heuristic Completion

```
1: Input:  $\mathcal{R}$  (remaining batsmen not yet placed)
2: Output: Ordered list of remaining batsmen
3: if  $\mathcal{R} = \emptyset$  then
4:   return  $[]$  (empty list)
5: end if
6: Initialize empty list:  $L \leftarrow []$ 
7: for each batsman  $b \in \mathcal{R}$  do
8:   if  $b \in \text{Batsman Profile}$  then
9:      $SR_b \leftarrow \text{Strike Rate of batsman } b \text{ from profile}$ 
10:   else
11:      $SR_b \leftarrow 100$  (default value)
12:   end if
13:   Append  $(b, SR_b)$  to  $L$ 
14: end for
15: Sort  $L$  by strike rate in descending order:  $L \leftarrow \text{sort}(L, \text{key} = SR, \text{desc})$ 
16: Extract batsman names: completion  $\leftarrow [b \mid (b, SR) \in L]$ 
17: return completion
```

ANALYSIS OF THE ALGORITHM

Let us consider the number of batsmen we operate on to be N

Here $N = 7$, $T = \text{number of overs} = 20$

As per algorithm 1 displayed in the previous slide:

- We are running the algorithm for N iterations and in each iteration i we iteration through an list of size $N-i$, selecting greedily and apply heuristic completion.
- The heuristic completion algorithm's complexity $R + R\log R = O(R\log R)$ where R is the number of remaining batsmen.
- So the overall time complexity is $O(N^3\log N)$.

($\sum N - i \approx N^2$ and we can consider heuristic completion's complexity as $N\log N$; time complexity $\Rightarrow O(N^3\log N + N^2T) = O(N^3\log N)$)

ALGORITHM OUTPUT

```
Position 1: V Kohli          → 68.67
Position 1: AB de Villiers   → 69.93
Position 1: MS Dhoni         → 68.67
Position 1: KA Pollard       → 70.02
Position 1: Shubman Gill     → 68.62
Position 1: KL Rahul         → 68.70
Selected: KA Pollard (70.02)
Position 2: V Kohli          → 70.75
Position 2: AB de Villiers   → 70.02
Position 2: MS Dhoni         → 70.75
Position 2: Shubman Gill     → 70.75
Position 2: KL Rahul         → 70.75
Selected: V Kohli (70.75)
Position 3: AB de Villiers   → 70.75
Position 3: MS Dhoni         → 70.08
Position 3: Shubman Gill     → 70.02
Position 3: KL Rahul         → 70.11
Selected: AB de Villiers (70.75)
Position 4: MS Dhoni         → 70.75
Position 4: Shubman Gill     → 70.75
Position 4: KL Rahul         → 70.75
Selected: MS Dhoni (70.75)
Position 5: Shubman Gill     → 70.75
Position 5: KL Rahul         → 70.75
Selected: Shubman Gill (70.75)
Position 6: KL Rahul         → 70.75
Selected: KL Rahul (70.75)
Position 7: R Sharma         → 70.75
Selected: R Sharma (70.75)
Final Order: ['KA Pollard', 'V Kohli', 'AB de Villiers', 'MS Dhoni', 'Shubman Gill', 'KL Rahul', 'R Sharma']
]
```

DP ALGORITHM

STATE VARIABLES

- S = subset of players not dismissed yet
- w = number of wickets lost
- t = current over number
- p = batsman on strike

SUBPROBLEM

$DP(S, w, t, p)$ is the maximum running that can be achieved when set of S batsmen are available, w wickets are down, t is current over and p is the batsman who is currently batting.

DP ALGORITHM

RECURRENCE RELATION

$$dp(S, w, t, p) = \max_{q \in S \setminus \{p\}} (\mathbb{E}[\text{runs}]_{(p,q)} + f_{(p,q)})$$

$$\mathbb{E}[\text{runs}]_{(p,q)} = \sum_{\text{bowlers}} P(\text{bowler}) \times \text{ExpectedRuns}(p, q, \text{bowler})$$

$$\begin{aligned} f_{(p,q)} = & \underbrace{(1 - P_{out})(1 - P_{rot}) \times dp(S, w, t + 1, p)}_{\text{Case 1: Not out, } p \text{ stays on strike}} \\ & + \underbrace{(1 - P_{out})P_{rot} \times dp(S, w, t + 1, q)}_{\text{Case 2: Not out, } q \text{ takes strike}} \\ & + \underbrace{P_{out} \times dp(S \setminus \{p\}, w + 1, t + 1, q)}_{\text{Case 3: Wicket falls}} \end{aligned}$$

p,q are the striker and non-striker respectively

P_{out} and P_{rot} are the probabilities of getting out and switching with non-strike for the current striker.

BASE CASE

The recursion stops (returns 0.0) when:

- $t \geq \text{num_overs}$ (Innings end)
- $w \geq \text{total_wickets}$ (All out)
- $|S| < 2$ (Not enough batsmen to form a pair)

ANALYSIS OF THE ALGORITHM

The state of the dynamic programming algorithm is defined by the tuple (S, w, t, p) . We calculate the total number of unique states by analyzing the cardinality of each component.

- **S (Remaining Batsmen):** Represents a subset of the original team N . The number of possible subsets is 2^N . Subsets suffice because the optimal future strategy depends solely on the set of currently available players, not the historical order of previous dismissals. This independence allows memoization to merge all $N!$ distinct permutation paths into 2^N unique resource states
- **p (Striker):** The current batsman on strike. Must be an element of S . In the worst case, there are N choices.
- **t (Over Number):** Ranges from 0 to T (where T is total overs, e.g., 20). Size is T .
- **w (Wickets Lost):** While explicitly tracked, w is mathematically redundant because it is determined by the size of S (i.e., $w = N_{\text{total}} - |S|$). Therefore, it does not multiply the state space complexity.

Total Number of States ($|S|$):

$$|S| \approx T \times N \times 2^N \quad (1)$$

ANALYSIS OF THE ALGORITHM

For every unique state, the algorithm calculates the recurrence relation to find the optimal decision. This involves iterating through possible partners and scenarios.

Let $N = |S|$ be the number of currently available batsmen.

1. **Outer Loop (Choosing Non-Striker q):** The algorithm iterates over all possible partners $q \in S \setminus \{p\}$. This gives $O(N)$ iterations.
2. **Inner Operations:** Inside the loop, we calculate:
 - **Scenario A (Survive):** Constant time lookup $O(1)$ from the dp array (memoization).
 - **Scenario B (Wicket Falls):** If a wicket falls, we must find the optimal *next* batsman n . This requires iterating over the remaining set $S \setminus \{p, q\}$. This takes $O(N)$ iterations.

The cost to compute one state is the Outer Loop $\times$ (Constant Cost + Inner Loop Cost).

$$\text{Cost}_{\text{state}} = O(N) \times (O(1) + O(N)) = O(N^2) \quad (2)$$

Metric	Complexity
State Space Size	$O(T \cdot N \cdot 2^N)$
Complexity Per State	$O(N^2)$
Total Time Complexity	$O(T \cdot N^3 \cdot 2^N)$

ALGORITHM OUTPUT

DP Score: 86.60

Optimal Policy Batting Order:

1. R Sharma
2. KA Pollard
3. KL Rahul
4. MS Dhoni
4. MS Dhoni
5. V Kohli
6. AB de Villiers
7. Shubman Gill

COMPARISON OF THE TWO ALGORITHMS

GREEDY ALGORITHM	DP ALGORITHM
Local Optimization: Make the decision that yields the maximum runs in the current over only.	Global Optimization: Make the decision that maximizes expected runs for the entire remainder of the innings.
Time Complexity: $O(N^3 \log N)$	Time Complexity = $O(TN^3 2^N)$
Restricts choices based on labels	Does not use labels; math determines who bats where.
Rigidly biases high Strike Rate players to bat earlier	Remembers optimal decisions for every possible game state and hence can find "unconventional" strategies
Optimizes the current batsman; assumes remaining partners are sorted by Strike Rate.	Evaluates specific pairs (Striker, Partner) running together

LOCAL OPTIMUM - BY GREEDY ALGORITHM

Greedy Score: 70.75

1. KA Pollard
2. V Kohli
3. AB de Villiers
4. MS Dhoni
5. Shubman Gill
6. KL Rahul
7. R Sharma

GLOBAL OPTIMUM - BY DP ALGORITHM

DP Score: 86.60

Optimal Policy Batting Order:

1. R Sharma
2. KA Pollard
3. KL Rahul
4. MS Dhoni
4. MS Dhoni
5. V Kohli
6. AB de Villiers
7. Shubman Gill



THANK YOU

GITHUB REPO LINK:

https://github.com/Humanoid2005/IPL_BatsmenOrdering_Optimiser